

Section E — Advanced Concepts Documentation

This section focuses on advanced SQL concepts such as conditional logic using CASE, understanding database reliability through ACID properties, and transaction management.

The main objective of this section is to practice:

- Conditional statements using CASE
- Conditional aggregation
- Understanding ACID properties
- Using transactions
- COMMIT and ROLLBACK operations

These concepts are important for maintaining data accuracy and consistency in database systems.

Database Selection

Query

```
USE shopease_db;
```

Explanation

This command selects the ShopEase database before running advanced SQL queries.

It ensures all queries execute in the correct database.

Q24. Classify products into price tiers using CASE

Query

```
SELECT  
    product_name,  
    unit_price,  
    CASE  
        WHEN unit_price < 1000 THEN 'Budget'  
        WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
```

```
        WHEN unit_price > 3000 THEN 'Premium'

    END AS price_tier

FROM products;
```

Explanation

This query uses the CASE statement to classify products into price categories.

Logic used:

- If price is less than 1000 → Budget
- If price is between 1000 and 3000 → Mid-Range
- If price is greater than 3000 → Premium

The result is displayed in a new column called price_tier.

CASE works like an if-else statement in SQL.

Purpose

Helps categorize products based on pricing.

Useful for product segmentation and pricing strategy.

Expected Output

Displays:

- product_name
- unit_price
- price_tier

Example:

Wireless Earbuds → Mid-Range

Running Shoes → Premium

Screenshot

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
9
10 -- =====
11 -- Q24. Classify products into price tiers using CASE
12 -- =====
13 • SELECT
14     product_name,
15     unit_price,
16
17     CASE
18         WHEN unit_price < 1000 THEN 'Budget'
19         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
20         WHEN unit_price > 3000 THEN 'Premium'

```

The result grid shows the following data:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget
Test Product	500.00	Budget

Q25. Count Delivered vs Not Delivered orders using CASE

Query

SELECT

SUM(

CASE

WHEN status = 'Delivered' THEN 1

ELSE 0

END

) AS delivered_orders,

SUM(

CASE

WHEN status != 'Delivered' THEN 1

ELSE 0

END

) AS not_delivered_orders

FROM orders;

Explanation

This query counts two categories of orders:

1. Delivered orders
2. Not Delivered orders

How it works:

- If status is Delivered, count as 1
- Otherwise count as 0

The SUM() function adds these values.

This is called conditional aggregation.

Purpose

Helps compare successful orders against pending, shipped, or cancelled orders.

Expected Output

Single row showing:

- delivered_orders
- not_delivered_orders

Screenshot

The screenshot shows a SQL IDE window titled 'shopease' with tabs for 'basic_queries', 'filtering_queries', 'aggregation_queries', 'joins_queries', and 'SQL File 7*'. The query editor displays the following SQL code:

```
25 -----
26 -- Q25. Count Delivered vs Not Delivered orders
27 -- Using CASE inside aggregate function
28 -----
29 SELECT
30     SUM(
31         CASE
32             WHEN status = 'Delivered' THEN 1
33             ELSE 0
34         END
35     ) AS delivered_orders,
36
37     SUM(
38         CASE
39             WHEN status != 'Delivered' THEN 1
```

The bottom of the window shows the 'Result Grid' with two columns: 'delivered_orders' and 'not_delivered_orders'. The 'delivered_orders' column has a value of 6, and the 'not_delivered_orders' column has a value of 4. The 'Result Grid' is currently in 'Read Only' mode.

Q26. ACID Properties Explanation

ACID properties are important rules that ensure reliable database transactions.

A — Atomicity

Meaning

A transaction is treated as a single unit.

Either all operations succeed or all fail.

Example

Bank transfer:

- ₹5000 deducted from Account A
- ₹5000 added to Account B

If the second step fails, the first step must also be undone.

This prevents incomplete transactions.

C — Consistency

Meaning

A transaction must keep the database valid before and after execution.

Example

If total bank balance was ₹10000 before transfer,
it should remain ₹10000 after transfer.

No money should be lost or created.

This ensures data correctness.

I — Isolation

Meaning

Multiple transactions should not affect each other.

Example

Two customers buying the same product at the same time should not cause incorrect stock calculations.

Each transaction must work independently.

This prevents concurrency issues.

D — Durability

Meaning

Once a transaction is committed, it remains saved permanently.

Example

If a bank transfer is completed and the system crashes immediately after, the transaction must still exist.

This ensures data is permanently stored.

Q27. Transaction Example

Query

```
START TRANSACTION;
```

```
INSERT INTO orders VALUES  
(1011, 102, CURDATE(), 'Pending', 1598.00);
```

```
INSERT INTO order_items VALUES  
(5016, 1011, 202, 1, 799.00, 0),  
(5017, 1011, 208, 1, 799.00, 0);
```

```
UPDATE products  
SET stock_qty = stock_qty - 1  
WHERE product_id = 202;  
UPDATE products
```

```
SET stock_qty = stock_qty - 1
```

```
WHERE product_id = 208;
```

```
COMMIT;
```

```
ROLLBACK;
```

Explanation

This transaction performs multiple related operations.

Step 1: Start Transaction

```
START TRANSACTION;
```

Begins the transaction block.

Step 2: Insert new order

Adds a new order into the orders table.

Step 3: Insert order items

Adds products purchased in that order.

Step 4: Update stock quantity

Reduces product stock after purchase.

For each product:

```
stock_qty = stock_qty - 1
```

Step 5: Commit

If all operations succeed:

```
COMMIT;
```

Saves all changes permanently.

Step 6: Rollback

If any step fails:

ROLLBACK;

Undoes all changes.

This ensures data consistency.

Purpose

Transactions ensure:

- all related operations complete together
- incomplete data is avoided
- database stays consistent

Expected Output

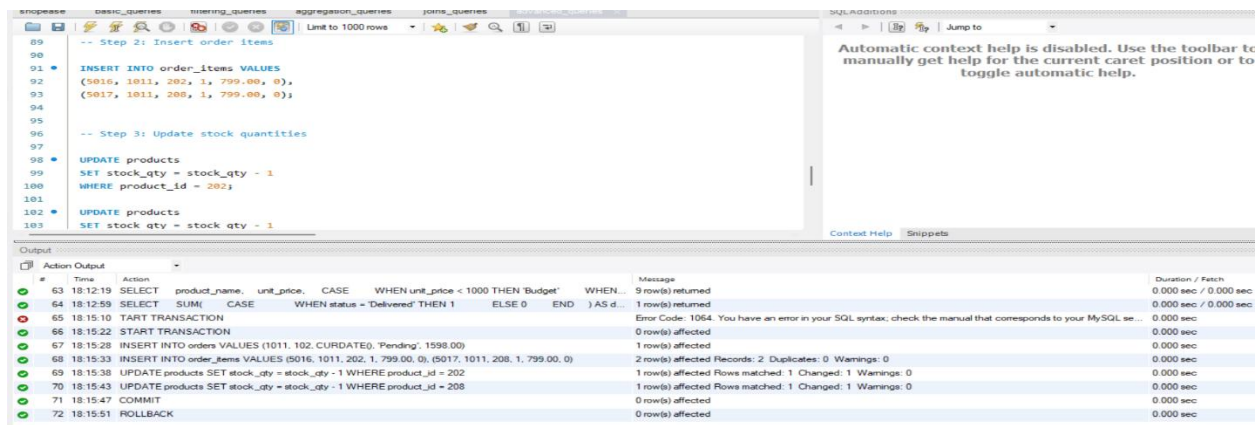
After COMMIT:

- New order added
- Order items inserted
- Stock updated

After ROLLBACK:

No changes saved.

Screenshot



Key Concepts Covered

- CASE statement
- Conditional aggregation
- ACID properties
- Transactions
- START TRANSACTION
- COMMIT
- ROLLBACK
- Data consistency

Summary

In this section, advanced SQL concepts were explored.

The queries demonstrated how conditional logic can be used for classification and counting.

The section also introduced transaction management and ACID properties, which are critical for ensuring reliable and consistent database operations.